קטע קוד A

```
 1. template<class E> class MyList {
 2.   class Node {
 3.     friend class MyList<E>;
 4.     E data;
 5.     Node* next = nullptr;
 6.   }; // end of class Node
 7.   Node* head = new Node;
 8.   Node* tail = head;
 9.   MyList(const MyList&) = default;
10. public:
11.   MyList() = default;
12.   MyList& operator=(const MyList&) = delete;
13.   void push_front(const E& data) {
14.     Node* node = new Node;
15.     node->data = data;
16.     node->next = head->next;
17.     if(head->next == nullptr) tail = node;
18.     head->next = node;
19.   }
20.
21.   void push_back(const E& data) {
22.     if(head->next == nullptr) {
23.         push_front(data); return;
24.     }
25.     MyList temp(*this);
26.     temp.head = temp.head->next;
27.     temp.push_back(data);
28.     temp.head = nullptr;
29.   }
30.
31.   ~MyList() {
32.     Node *node = head, *next;
33.     while(node != nullptr) {
34.         next = node->next;
35.         delete node;
36.         node = next;
37.     }
38.   }
39.
40.   E first() {
41.       return head->data;
42.   }
43.
44.   template<class Function>
45.   void apply (Function f) const {
46.     Node* node = head->next;
47.     while(node != nullptr) {
48.         f(node->data);
```

```
        node = node->next;
    }
  }
};

class Num {
protected:
    int num;
public:
    Num(int num) : num(num) { }
    void print() const { cout << num << endl; }
    virtual operator int() const { return num; }
    virtual ~Num() {}
};

class NumNum: public Num {
public:
    NumNum(int num=0): Num(num + 1) { }
    operator int() const { return num * num; }
};

int main() {
    MyList<int> m1;
    // m1.push_back(NumNum());
    // cout << m1.first() << endl;

    for(int i = 1; i <= 2; ++i) m1.push_front(i);
    for(int i = 1; i <= 2; ++i) m1.push_back(i);
    m1.apply( [](const auto& val){ cout << val << endl; } );

    MyList<NumNum> m2;
    // MyList<Num> m2;
    m2.push_back(4);
    m2.push_front(9);
    m2.apply([](const auto& val){ val.print(); });

    MyList<Num*> m3;
    for(int i = 1; i <= 3; ++i) m3.push_front(new NumNum(i));
    m3.apply([](const auto& val){ val->print(); });
    // m3.apply([](const auto& val){ cout << *val << endl; });
    m3.apply([](auto& val){ delete val; });
}
```

קטע קוד B

```
 1. using Groups = map<std::string, size_t>;
 2.
 3. class MyString {
 4.     static Groups groups;
 5.     vector<Groups::iterator> substrings;
 6.
 7.     static Groups::iterator joinGroup(const std::string& s) {
 8.         auto itr = groups.find(s);
 9.         if (itr == groups.end()) {
10.             // See explanation regarding map::insert at line 109
11.             itr = groups.insert({s, 1}).first;
12.         } else {
13.             ++(itr->second);
14.         }
15.         return itr;
16.     }
17.
18.     static void leaveGroups(const vector<Groups::iterator>& subs) {
19.         for (auto itr : subs) {
20.             if (--(itr->second) == 0) groups.erase(itr);
21.         }
22.     }
23.
24.     static void addOneToAll(const vector<Groups::iterator>& subs) {
25.         for (auto itr : subs) {
26.             ++(itr->second);
27.         }
28.     }
29.
30. protected:
31.     string original;
32.
33. public:
34.     static size_t getCount(const std::string& s) {
35.         size_t count = groups[s];
36.         cout << "\"" << s << "\" (" << count << ")\n";
37.         return count;
38.     }
39.     static void printGroups() {
40.         for (auto g : groups) {
41.             cout << "\"" << g.first << "\" (" << g.second << ")\n";
42.         }
43.     }
44.     MyString(const char* s): original(s) {
45.         substrings.push_back(joinGroup(s));
46.     }
47.
```

```
    ~MyString() { leaveGroups(substrings); }

    MyString(const MyString& s) : substrings(s.substrings) {
        addOneToAll(substrings);
    }

    MyString& operator=(const MyString& s) {
        if (this != &s) {
            leaveGroups(substrings);
            substrings = s.substrings;
            addOneToAll(substrings);
        }
        return *this;
    }

    MyString& operator+=(const MyString& s) {
        // Below we insert all elements in s.substrings
        // into the end of this->substrings
        // (i.e., concatenating s.substrings into this->substrings)
        substrings.insert(
            /* insert before: */ substrings.end(),
            /* a copy of all elements in the range: */
            s.substrings.begin(), s.substrings.end()
        );

        // Now count me in for all the substrings that were added
        addOneToAll(s.substrings);
        return *this;
    }

    virtual string original_str() const {
        return original;
    }

    friend ostream& operator<<(ostream& out, const MyString& s) {
        for (auto itr : s.substrings) out << itr->first;
        return out;
    }
};

class MyString2 : public MyString {
public:
    MyString2(const char* s): MyString(s) { }
    virtual string original_str() const override {
        return string("MyString2: ") + original;
    }
};
```

```
 95. class MyString3 : public MyString2 {
 96. public:
 97.     MyString3(const char* s): MyString2(s) { }
 98.     virtual string original_str() {
 99.         return string("MyString3: ") + original;
100.     }
101.     ~MyString3() {
102.         cout << "~" << original_str() << endl;
103.     }
104. };
105.
106. // in MyString.cpp
107. Groups MyString::groups;
108.
109. // Explanation regarding map::insert (line 11):
110. // map.insert returns std::pair<iterator, bool>
111. // 'first' is an iterator pointing to the new item or
112. // an item with the same key that already existed, which
113. // prevented the insertion
114. // 'second' is whether insertion was performed
115.
116. int main() {
117.     // MyString::getCount("Hello");
118.     MyString::printGroups();
119.
120.     vector<MyString> vec = {"Hello", "World"};
121.     vec[0] += "World";
122.     // vec.resize(5);
123.     MyString::printGroups();
124.
125.     MyString* s1 = new MyString3("Hello");
126.     cout << s1->original_str() << endl;
127.
128.     MyString s3 = *s1;
129.     delete s1;
130.     vec[1] += vec[0];
131.     vec[0] = "!!!";
132.     // cout << s3 << endl;
133.     MyString::printGroups();
134. }
```

קטע קוד C – לשאלות הבונוס

```
 1. // Strings, counted in a map
 2. using Groups = map<std::string, size_t>;
 3.
 4. class MyString {
 5.     static Groups groups;
 6.     list<Groups::iterator> substrings;
 7.
 8.     static Groups::iterator joinGroup(const std::string& s) {
 9.         auto itr = groups.find(s);
10.         if (itr == groups.end()) {
11.             // See explanation regarding map::insert at line 68
12.             itr = groups.insert({s, 1}).first;
13.         } else {
14.             ++(itr->second);
15.         }
16.         return itr;
17.     }
18.
19.     static void leaveGroups(const list<Groups::iterator>& subs) {
20.         for (auto itr : subs) {
21.             if (--(itr->second) == 0) groups.erase(itr);
22.         }
23.     }
24.
25.     static void addOneToAll(const list<Groups::iterator>& subs) {
26.         for (auto itr : subs) {
27.             ++(itr->second);
28.         }
29.     }
30.
31.     public:
32.         static void printGroups() {
33.            for(const auto& g : groups)
34.              cout << '<' << g.first << "> (" << g.second << ")\n";
35.         }
36.         MyString(const char* s = "") {
37.             substrings.push_back(joinGroup(s));
38.         }
39.         ~MyString() { leaveGroups(substrings); }
40.         MyString(const MyString& s): substrings(s.substrings) {
41.             addOneToAll(substrings);
42.         }
43.         MyString(MyString&& s): substrings(std::move(s.substrings)) {}
44.         MyString& operator=(MyString s) {
45.             leaveGroups(substrings);
46.             substrings = std::move(s.substrings);
47.             return *this;
48.         }
```

```
        friend ostream& operator<<(ostream& out, const MyString& s) {
            for(auto itr: s.substrings)
                out << itr->first;
            return out;
        }

        MyString& operator+=(MyString s) {
            // The line below *moves* all elements in s.substrings
            // to the end of this->substrings
            substrings.splice(
                /*insert to my end:*/ substrings.end(),
                /*elements from:*/ s.substrings,
                /*in the range:*/ s.substrings.begin(),
                s.substrings.end()
            );
            return *this;
        }
};

// in MyString.cpp
Groups MyString::groups;

int main() {
    MyString arr[3] = {};
    arr[1] = "Good";
    MyString::printGroups();

    MyString* s2 = new MyString;
    arr[0] = arr[2] = "Luck";
    std::swap(arr[1], *s2);
    cout << arr[1] << endl;
    vector<MyString> vec(arr+1, arr+3);
    MyString::printGroups();

    MyString s3;
    s3 += arr[0];
    s3 += arr[1];
    s3 += arr[2];
    cout << s3 << endl;

    delete s2;
    MyString::printGroups();

    return 0;
}
```

בהצלחה !